

ARM PrimeCell Driver

VIC v1.0 (PL192)
API Specification

PrimeCell Driver API Specification

© Copyright ARM Limited 2002. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- the document title
- the documents number
- the page number(s) to which your comments refer
- a concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	F
PrimeCell driver identifier	VIC
Code Version	1.0
PrimeCell identifier(s)	PL192

Contents

1	ABOUT THIS DOCUMENT	6
1.1	Change Control	6
1.1.1	Change history	6
2	SUMMARY	7
3	FEATURES	7
4	DRIVER USAGE	8
4.1	Overview	8
4.2	Configuring the Interrupt Controller	8
4.2.1	apINT_VERSION	9
4.2.2	apCONFIG_CORE_SUPPORTS	9
4.2.3	apOS_CONFIG_INT_REENTRANT	9
4.2.4	apOS_CONFIG_VIC_USE_PREDISPATCH_CODE	9
4.2.5	apOS_CONFIG_VIC_USE_POSTDISPATCH_CODE	9
4.2.6	apOS_CONFIG_VIC_SINGLE_FIQ	10
4.2.7	apOS_CONFIG_INT_USE_ISA	10
4.2.8	apOS_CONFIG_INT_ISR_ISA	10
4.2.9	apOS_INT_CONFIG_SOURCES_USED	10
4.2.10	apOS_CONFIG_VIC_AT_0xFFFFF000	10
4.2.11	apOS_CONFIG_VIC_0_BLOCKING_MODE	10
4.2.12	apOS_CONFIG_VIC_NUMBER	10
4.3	Initialisation	11
4.3.1	System Initialisation	11
4.3.2	Peripheral Drivers Initialisation	11
4.4	Interrupt Handling	11
5	DETAILS	12
6	RELEASED FILES	12
7	PUBLIC API	13
7.1	Type Definitions	13
7.1.1	apVIC_eError	13
7.1.2	apVIC_eInterruptType	13
7.1.3	apVIC_rDispatchHandler	13
7.1.4	apVIC_sInitialData	14
7.1.5	apVIC_IRQ_LOWEST_PRIORITY	14
7.1.6	apVIC_FIQ_LOWEST_PRIORITY	14

7.2	External Functions	14
7.2.1	apVIC_Initialize	14
7.2.2	apVIC_HandlerSet	15
7.2.3	apVIC_RawHandlerSet	16
7.2.4	apVIC_HandlerClear	16
7.2.5	apVIC_PrioritySet	17
7.2.6	apVIC_PriorityGet	17
7.2.7	apVIC_PriorityMaskSet	18
7.2.8	apVIC_PriorityMaskGet	18
7.2.9	apVIC_PriorityDaisySet	19
7.2.10	apVIC_PriorityDaisyGet	19
7.2.11	apVIC_IRQPreDispatchSet	20
7.2.12	apVIC_IRQPostDispatchSet	20
7.2.13	apVIC_ControllerDisable	20
7.2.14	apVIC_ControllerRestore	21
7.2.15	apVIC_InterruptClear	21
7.2.16	apVIC_InterruptDisable	21
7.2.17	apVIC_InterruptEnable	22
7.2.18	apVIC_ProgrammedInterruptSet	22
7.2.19	apVIC_ProgrammedInterruptClear	22
7.2.20	apVIC_GetInterruptStatus	23
7.2.21	apVIC_SingleFIQSourceSet	23
7.2.22	apVIC_SingleFIQSourceClear	24

1 ABOUT THIS DOCUMENT

1.1 Change Control

1.1.1 Change history

Issue	Date	Change
A01	30 th , April, 2002	Draft issue of API
A02	22 nd , May, 2002	Updated for review comments
A03	4 th , September, 2002	API version v1.0
B	1 st , October, 2002	Next release of API after review
C	9 th , October, 2002	Next release of API after review
D	9 th , October, 2002	Driver ROM and RAM size added
E	21 st , October, 2002	Explanations of sections 5, 6, 7 added
F	23 rd , October, 2002	The lowest and the highest values of interrupt priority changed Type definition of apVIC_LOWEST_PRIORITY deleted

2 SUMMARY

The PrimeCell *Vectored Interrupt Controller* (VIC) is an *Advanced Microcontroller Bus Architecture* (AMBA) compliant, *System-on-Chip* (SoC) peripheral that is developed, tested, and licensed by ARM.

The Vectored Interrupt Controller has been designed to improve interrupt latency. It is intended to make the process of handling multiple interrupt sources faster. It achieves this by supplying the address of the correct interrupt handler for the interrupt source according to a pre-set priority.

3 FEATURES

The main features supported by the PrimeCell VIC driver are:

- Supports for 32 vectored IRQ interrupts.
- Priority can be set for individual interrupt source.
- Priority can be set for the daisy-chained interrupt source.
- Sets interrupt type to either IRQ (vectored and Raw) or FIQ.
- Software interrupt generation.
- Supports Raw interrupt handler which will further shorten response latency of IRQ interrupts.
- Supports interrupt masking.
- Supports priority level masking.
- Supports 32 software interrupt priority levels for non-vectored FIQ interrupt sources.
- Supports all daisy-chained interrupt controllers in the system.
- Supports two types of daisy-chained interrupt controller connectivity – VIC 0 blocking and Daisy-chained VIC blocking modes.

4 DRIVER USAGE

4.1 Overview

The driver has been designed such that there are two types of functions: those designed to be called by the central system at initialisation:

- `apVIC_Initialize()`

and those designed to be called by individual peripheral drivers during their initialisation and operation:

- `apVIC_HandlerSet()`
- `apVIC_HandlerClear()`
- `apVIC_RawHandlerSet()`
- `apVIC_PrioritySet()`
- `apVIC_PriorityGet()`
- `apVIC_PriorityDaisySet()`
- `apVIC_PriorityDaisyGet()`
- `apVIC_PriorityMaskSet()`
- `apVIC_PriorityMaskGet()`
- `apVIC_InterruptEnable()`
- `apVIC_InterruptDisable()`
- `apVIC_InterruptClear()`
- `apVIC_HandlerClear()`
- `apVIC_ControllerDisable()`
- `apVIC_ControllerRestore()`
- `apVIC_ProgrammedInterruptSet()`
- `apVIC_ProgrammedInterruptClear()`
- `apVIC_IRQPreDispatchSet()`
- `apVIC_IRQPostDispatchSet()`
- `apVIC_GetInterruptStatus()`
- `apVIC_SingleFIQSourceSet()`
- `apVIC_SingleFIQSourceClear()`

4.2 Configuring the Interrupt Controller

The VIC interrupt controller can be configured through the constants in **`apintcfg.h`**, which is in directory **`apos`**.

Note that the format of this file is unusual, as it is included by both assembler and C files. The main difference is that the preprocessor directive '#define' must be written with a space after the '#' character.

4.2.1 apINT_VERSION

This specifies the version of the interrupt controller. Where a PrimeCell module specifies this as the PrimeCell number (e.g. 11 for a UART PL011), the interrupt controller uses one of the options below:

- ◆ apVERSION_INTEGRATOR indicates that the behaviour matches the controller on the Integrator board.
- ◆ apVERSION_ARMULATOR indicates that the behaviour matches the ARMulator simulation.
- ◆ apVERSION_VECTORED indicates that a PL190 Vectored Interrupt Controller (VIC) only will be used.
- ◆ apVERSION_VIC_ON_LM indicates that the Integrator development platform is being used with a Vectored Interrupt Controller on a Logic Module in addition to the standard Integrator controller.
- ◆ apVERSION_VIC_ON_FQ indicates that the Integrator development platform is being used with a Vectored Interrupt Controller on a Logic Module in addition to the standard Integrator controller, but with nVICFIQ output pin connected to ARM FIQ input, and nVICIRQ output remained unconnected.
- ◆ apVERSION_VECTORED_PL192 indicates that a PL192 Vectored Interrupt Controller only will be used.
- ◆ apVERSION_VIC_PL192_ON_LM indicates that the Integrator development platform is being used with a PL192 Vectored Interrupt Controller on a Logic Module in addition to the standard Integrator controller.

4.2.2 apCONFIG_CORE_SUPPORTS

This should be defined as one of the following:

- ◆ apCONFIG_ARM_AND_THUMB indicates that the core can support both ARM and Thumb states and the assembler code is then compiled for inter-working.
- ◆ apCONFIG_ARM_ONLY indicates that the core will only support ARM code (e.g. StrongARM).

4.2.3 apOS_CONFIG_INT_REENTRANT

If defined as 1, driver will support re-entrant interrupt sources.

Note. Because of the hardware implementation only up to 16 re-entrant interrupt sources can be used in the system. In VIC 0 blocking mode re-entrant interrupt handlers cannot be used for daisy-chained sources.

4.2.4 apOS_CONFIG_VIC_USE_PREDISPATCH_CODE

- ◆ If defined as 1, the Interrupt controller will execute a specified routine before calling each IRQ Interrupt Service Routine.
- ◆ If defined as 0, no pre-dispatch routine is available, but there will be a slight speed gain.

This routine is specified using apVIC_IRQPreDispatchSet() function.

4.2.5 apOS_CONFIG_VIC_USE_POSTDISPATCH_CODE

- ◆ If defined as 1, the Interrupt controller will execute a specified routine after calling each IRQ Interrupt Service Routine.
- ◆ If defined as 0, no post-dispatch routine is available, but there will be a slight speed gain.

This routine is specified using apVIC_IRQPostDispatchSet() function.

4.2.6 apOS_CONFIG_VIC_SINGLE_FIQ

- ◆ If defined as 1, the driver will check for non-zero value of the pointer for a source which is marked as the only active registered interrupt source for FIQ. If this pointer is non-zero then this entry will always be loaded and the ISR called whenever an FIQ exception occurs. If the table lookup is required (if there is more than one FIQ handler, or if interrupt source checking is required) then define this switch as 0 or set the pointer to NULL by calling apVIC_SingleFIQSourceClear() function.

4.2.7 apOS_CONFIG_INT_USE_ISA

- ◆ If defined as apCONFIG_THUMB_ONLY, veneers will be added to routines in **oss.s** to make calling from Thumb code more efficient.
- ◆ If defined as apCONFIG_ARM_ONLY, veneers will be added by the linker when inter-working as usual.

4.2.8 apOS_CONFIG_INT_ISR_ISA

- ◆ If defined as apCONFIG_ARM_ONLY, interrupt service routines are expected to be in ARM code. The linker will add veneers when inter-working with Thumb ISRs as usual.
- ◆ If defined as apCONFIG_THUMB_ONLY, interrupt service routines are expected to be in Thumb code. The linker will add veneers when inter-working with ARM ISRs as usual.

4.2.9 apOS_INT_CONFIG_SOURCES_USED

Each interrupt controller can control up to 32 interrupt sources. By default, enough space is assigned for this many sources. However, if a lesser number of sources are required, this may be specified by defining the constant apOS_INT_CONFIG_SOURCES_USED. This constant should specify the total number of interrupt sources used across all interrupt controllers as below:

```
# define apOS_INT_CONFIG_SOURCES_USED 16
```

4.2.10 apOS_CONFIG_VIC_AT_0xFFFFF000

In a real system, a Vectored Interrupt Controller (if present) should be placed at 0xFFFFF000 in the memory map, although this may not be possible in a development system. This will mean that a dispatcher need not be called for interrupts, so saving a number of cycles.

This is indicated by setting this constant as below:

```
# define apOS_CONFIG_VIC_AT_0xFFFFF000 1
```

4.2.11 apOS_CONFIG_VIC_0_BLOCKING_MODE

This defines the type of daisy-chained interrupt controller connectivity. Define as 1 for VIC 0 blocking and as 0 for Daisy-chained VIC blocking mode.

4.2.12 apOS_CONFIG_VIC_NUMBER

This defines the number of Vectored Interrupt Controllers used in the system.

4.3 Initialisation

4.3.1 System Initialisation

The function `apVIC_Initialize()` must be called before any of the other functions in this module (or any of the other interrupt driven peripheral drivers). This call sets the defaults for the interrupt module, and resets the internal state for the drivers.

4.3.2 Peripheral Drivers Initialisation

When individual peripheral wants to use VIC, each peripheral driver should set up corresponding interrupt handler, interrupt type and the priority of this interrupt by calling the following three functions:

- `apVIC_HandlerSet()` (IRQ vectored interrupts and FIQ non-vectored interrupts) or `apVIC_RawHandlerSet()` (for IRQ Raw interrupts)
- `apVIC_PrioritySet()`
- `apVIC_InterruptEnable()`

If the priority and interrupt type are the same as the default set in `apVIC_Initialize()`, the second step can be skipped. The interrupt types that will be assigned to interrupt sources are Raw, IRQ vectored or FIQ non-vectored. This assignment can be changed by calling `apVIC_PrioritySet()` function with specified interrupt type.

4.4 Interrupt Handling

If normal interrupt handler is performed (`apVIC_HandlerSet()` was called), it should have two arguments. The first one should be ignored and is kept in the argument list only for the correct passing of the second one. The second one is the number associated with device generating the interrupt. This will be useful when there are several interrupt sources sharing the same interrupt handler. This number is set when calling `apVIC_HandlerSet()`. However, when IRQ Raw ISR handling is performed (`apVIC_RawHandlerSet()` was called), there will be no arguments passed to the handler. It will be the handler's responsibility to find out which device has generated the interrupt.

During the handling of an interrupt, the peripheral driver should:

- call `apVIC_InterruptDisable()`
- handle the interrupt in the peripheral to cancel the interrupt source
- call `apVIC_InterruptEnable()`, to re-enable the interrupt again
- call `apVIC_InterruptClear()` in Raw IRQ interrupt handler

Please note, that `apVIC_InterruptClear()` should be called only at the end of Raw IRQ interrupt handlers.

5 DETAILS

This section contains some details of PL192 driver for the specified configuration.

◆ Peak performance obtained in tests:	N/A
◆ ROM size (RO code + RO data) (release build):	3920*
◆ RAM size (ZI data) per instance (release build):	1512*
◆ Does this driver support multiple PrimeCells?:	NO
◆ Can these be detected at run-time?	N/A
◆ Does this driver support both endian-nesses? ¹	YES
◆ Does this driver support Thumb? ²	YES

* Configuration:

• # define apOS_CONFIG_INT_REENTRANT	0
• # define apOS_CONFIG_VIC_USE_PREDISPATCH_CODE	0
• # define apOS_CONFIG_VIC_USE_POSTDISPATCH_CODE	0
• # define apOS_CONFIG_VIC_SINGLE_FIQ	0
• # define apOS_CONFIG_VIC_0_BLOCKING_MODE	1
• # define apOS_CONFIG_VIC_NUMBER	1

6 RELEASED FILES

This section contains the list of all released files for PL192 driver.

PL192-ISPC-1000-F	This API document
PL192-TREP-1000	Test Report document
apPL192/appl192.h	Public header file
apPL192/pl192.h	Private header file
apPL192/pl192wrappers.s	Driver source code
apPL192/pl192dispatch.s	Driver source code
apPL192/pl192.c	Driver source code
apPL192/pl192tst.c	Self-test C module
apPL192/tstpl192.s	Self-test assembler module

1 To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

2 The driver may include ARM code, but this must interwork with a Thumb build.

7 PUBLIC API

This section contains description of all public type definitions and functions of PL192 driver.

7.1 Type Definitions

7.1.1 apVIC_eError

VIC specific error return codes. Errors will be returned typecast to the general error type apError.

Declaration

```
typedef enum apVIC_eError
```

Elements

apERR_VIC_BAD_INT_TYPE	Error interrupt type of interrupt source
apERR_VIC_BAD_CALL	Function call whilst interrupt is being serviced
apERR_VIC_SOURCE_NOT_FOUND	Interrupt source was not found
apERR_VIC_BAD_PRIORITY	Error priority parameter

7.1.2 apVIC_eInterruptType

Enumeration of the interrupt types the VIC can generate.

Declaration

```
typedef enum apVIC_eInterruptType
```

Elements

apVIC_IRQ_VECTORED	Vectored IRQ exception
apVIC_IRQ_RAW	Vectored IRQ Raw exception
apVIC_FIQ	Non-vectored FIQ exception

7.1.3 apVIC_rDispatchHandler

Function pointer type for use with apVIC_PreDispatchSet() and apVIC_PostDispatchSet().

Declaration

```
typedef void (apVIC_rDispatchHandler) (void)
```

7.1.4 apVIC_sInitialData

Structure for passing the default interrupt priority into the interrupt initialisation function apVIC_Initialize(). No data passed in apVIC_sInitialData structure.

Declaration

```
typedef void apVIC_sInitialData
```

7.1.5 apVIC_IRQ_LOWEST_PRIORITY

The value which defines the lowest interrupt hardware priority for IRQ sources.

Declaration

```
#define apVIC_IRQ_LOWEST_PRIORITY 15
```

7.1.6 apVIC_FIQ_LOWEST_PRIORITY

The value which defines the lowest interrupt software priority for FIQ sources.

Declaration

```
#define apVIC_FIQ_LOWEST_PRIORITY 31
```

7.2 External Functions

7.2.1 apVIC_Initialize

Initialises the interrupt system, configures all the interrupt sources to the same default IRQ vectored type with the lowest priority and specifies the base address of the interrupt hardware registers.

Declaration

```
PUBLIC apError apVIC_Initialize( apOS_VIC_oId          oId,  
                               apOS_System_eBaseAddress eBase,  
                               UWORD32                NumberInterrupts,  
                               CONST apOS_INT_oInterruptSource *pSources,  
                               apVIC_sInitialData      *pInitial )
```

Parameters

<i>old</i>	The interrupt controller to initialise
<i>eBase</i>	The base address of the interrupt hardware registers
<i>NumberInterrupts</i>	Pass as NULL
<i>pSources</i>	Pass as NULL

pInitial

Pass as NULL

Return Value

apERR_NONE

The interrupt controller initialised, no errors

apERR_BAD_PARAMETER

The specified controller is out of the current implementation's range

Remarks

The specified base address is remembered for all future accesses to the interrupt hardware using this API. All interrupt sources are disabled, by masking out the interrupts. All interrupt handlers are removed and reset to a default handler which does nothing inside. All interrupts are assigned to IRQ by default, and with the default lowest priority.

7.2.2 apVIC_HandlerSet

Sets an interrupt handling function for a specified interrupt source. Also registers an ID number (or instance) which will be passed to the interrupt handler when the interrupt occurs.

Declaration

```
PUBLIC apError apVIC_HandlerSet( CONST apOS_INT_oInterruptSource oSource,  
                                apOS_INT_rServiceRoutine          rHandler,  
                                UWORD32                          Parameter )
```

Parameters

oSource

The interrupt source to attach the handler to

rHandler

The interrupt handling function

Parameter

Parameter to be associated with this interrupt source and passed to the handler function when the interrupt occurs

Return Value

apERR_NONE

Handler set, no errors

apERR_VIC_BAD_CALL

The function was called whilst interrupt is being serviced

apERR_VIC_SOURCE_NOT_FOUND

Free interrupt source was not found

Remarks

Only one handler can be assigned to each source, and if a second is assigned to a source, then the first will be removed, and then the second added.

The handler will be assigned to vectored IRQ source and interrupt priority will be set to the default lowest value.

The ID number specified to be associated with the interrupt source is designed to allow the same interrupt handling function to be used for multiple interrupt sources (for example in a system where there are several identical timers). The ID number is passed to the interrupt handling function (as the first parameter, in R0) to allow it to know which of the interrupt sources has generated the interrupt.

Note: *Always set a handling function before enabling the corresponding interrupt source. This function must not be called from an interrupt handler.*

7.2.3 apVIC_RawHandlerSet

Sets an IRQ Raw interrupt handling function for a specified interrupt source in order to speed up the interrupt handling response. This function will not pass ID number to the handler so that can avoid extra stacking cycles.

Declaration

```
PUBLIC apError apVIC_RawHandlerSet( CONST apOS_INT_oInterruptSource oSource,  
                                   apOS_INT_rRawISR                rRawHandler )
```

Parameters

<i>oSource</i>	The interrupt source to attach the handler to
<i>rRawHandler</i>	The raw interrupt handling function

Return Value

<i>apERR_NONE</i>	Raw handler set, no errors
<i>apERR_VIC_BAD_CALL</i>	The function was called whilst interrupt is being serviced
<i>apERR_VIC_SOURCE_NOT_FOUND</i>	Free interrupt source was not found

Remarks

Only one handler can be assigned to each source, and if a second is assigned to a source, then the first will be removed, and then the second set up. The function will return error code if free interrupt source was not found.

Note: *Always set a handling function before enabling the corresponding interrupt source. This function must not be called from an interrupt handler. Do not use this function to set Raw Interrupt Handler when user wants the interrupt be re-entrant unless user writes his own appropriate stacking operation at the beginning of their Raw handler function.*

7.2.4 apVIC_HandlerClear

Removes the interrupt handling function for a specified interrupt source and resets to the default handler, which does nothing inside but returns from the interrupt.

Declaration

```
PUBLIC apError apVIC_HandlerClear( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

<i>oSource</i>	The interrupt source to remove the handler from
----------------	---

Return Value

<i>apERR_NONE</i>	Handler cleared, no error
<i>apERR_VIC_SOURCE_NOT_FOUND</i>	Interrupt source was not found

Remarks

As only one handler can be assigned to each source, removing it will leave a default handler for a particular interrupt. Therefore, this routine also disables the interrupt by calling the function `apVIC_InterruptDisable()`. For all registered interrupts this function also resets the interrupt type of this source to vectored IRQ. Interrupt priority is set to the default lowest value.

7.2.5 apVIC_PrioritySet

Selects whether the specified interrupt source would either produce a vectored IRQ, Raw IRQ or FIQ non-vectored interrupt exception. Also records the priority level of the interrupt, this is used to determine which interrupts get serviced first if there are multiple outstanding active interrupt sources.

Declaration

```
PUBLIC apError apVIC_PrioritySet(  CONST ap0S_INT_oInterruptSource  oSource,
                                apVIC_eInterruptType             eType,
                                UBYTE8                           Priority )
```

Parameters

<i>oSource</i>	The interrupt source to set priority
<i>eType</i>	Which exception to produce, either apVIC_IRQ_Vectored, apVIC_IRQ_Raw or apVIC_FIQ
<i>Priority</i>	The interrupt priority, integer from 0 (highest priority) to 15 (lowest priority) for IRQ interrupts (hardware priority) and from 0 to 31 for FIQ interrupts (software priority)

Return Value

<i>apERR_NONE</i>	Priority set, no error
<i>apERR_VIC_BAD_CALL</i>	The function was called whilst interrupt is being serviced
<i>apERR_VIC_SOURCE_NOT_FOUND</i>	Interrupt source was not found
<i>apERR_VIC_BAD_INT_TYPE</i>	Different interrupt type was attempted to set up for Raw interrupt source
<i>apERR_VIC_BAD_PRIORITY</i>	Priority parameter is out of range

Remarks

Selects the interrupt type for this interrupt source, the default (if this function is not called) is that the interrupt is assumed to be a vectored IRQ, with the default lowest priority.

Note: *This function does not enable the interrupt. To enable the interrupt, call the function apVIC_InterruptEnable(). This causes an update of the interrupt dispatch table during which all interrupts will be temporarily disabled. This function must not be called from an interrupt handler.*

For daisy-chained IRQ sources the priority is used for interrupt level masking and does not affect their interrupt servicing priority, which has the same value for all sources and is defined by the value set up by apVIC_PriorityDaisySet() function.

7.2.6 apVIC_PriorityGet

Returns interrupt type and priority of the specified interrupt source.

Declaration

```
PUBLIC apError apVIC_PriorityGet(  CONST ap0S_INT_oInterruptSource  oSource,
                                apVIC_eInterruptType             *pType,
                                UBYTE8                           *pPriority )
```

Parameters

<i>oSource</i>	The interrupt source to read interrupt type and priority
<i>pType</i>	Pointer to the variable where the interrupt type will be written to
<i>pPriority</i>	Pointer to the variable where the priority will be written to

Return Value

<i>apERR_NONE</i>	Interrupt type and priority read, no errors
<i>apERR_VIC_SOURCE_NOT_FOUND</i>	Interrupt source was not found

Note: For daisy-chained sources this function returns the interrupt priority used for interrupt level masking. The interrupt servicing priority can be obtained by *apVIC_PriorityDaisyGet()* function.

7.2.7 apVIC_PriorityMaskSet

Sets up the mask for the interrupt priority level for IRQ sources.

Declaration

```
PUBLIC apError apVIC_PriorityMaskSet( apOS_VIC_oId oId,
                                     UHWD16      Mask )
```

Parameters

<i>old</i>	The interrupt controller to set up priority levels mask
<i>Mask</i>	The mask value for interrupt priority levels

Return Value

<i>apERR_NONE</i>	Interrupt priority levels mask set up, no error
<i>apERR_BAD_PARAMETER</i>	The specified controller is out of the current implementation's range

Remarks

Each bit in mask value corresponds with the appropriate IRQ priority level, i.e. bit 0 – priority level 0 (highest), bit 15 – priority level 15 (lowest), HIGH set bits enable and set to LOW disable the priority level. If all bits in mask are set to LOW value, then no interrupts will be able to generate an IRQ. The default value set up by *apVIC_Initialize()* function is 0xFFFF, which enables all priority levels.

7.2.8 apVIC_PriorityMaskGet

Returns the current mask value for the interrupt priority level for IRQ sources.

Declaration

```
PUBLIC apError apVIC_PriorityMaskGet( apOS_VIC_oId oId,
                                     UHWD16      *pMask )
```

Parameters

<i>old</i>	The interrupt controller to read the priority levels mask
<i>pMask</i>	Pointer to the variable the mask for interrupt priority levels to be read in

Return Value

<i>apERR_NONE</i>	Interrupt priority levels mask read, no error
<i>apERR_BAD_PARAMETER</i>	The specified controller is out of the current implementation's range

7.2.9 apVIC_PriorityDaisySet

Sets up the priority level for daisy-chained interrupt sources.

Declaration

```
PUBLIC apError apVIC_PriorityDaisySet( apOS_VIC_oId oId,
                                     UBYTE8      Priority )
```

Parameters

<i>old</i>	The interrupt controller to set up priority for daisy-chained sources
<i>Priority</i>	The interrupt priority, integer from 0 (highest priority) to 15 (lowest priority)

Return Value

<i>apERR_NONE</i>	Interrupt priority set up, no error
<i>apERR_BAD_PARAMETER</i>	The specified controller is out of the current implementation's range
<i>apERR_VIC_BAD_PRIORITY</i>	Priority parameter is out of range
<i>apERR_VIC_BAD_CALL</i>	In VIC 0 blocking mode there was an attempt to change priority to the higher value whilst servicing an interrupt

Note: *In VIC 0 blocking mode (apOS_CONFIG_VIC_0_BLOCKING_MODE in **apintcfg.h** is # defined to 1) this function must not be called from an interrupt handler with priority to be set higher than the current set value.*

7.2.10 apVIC_PriorityDaisyGet

Returns the priority level for daisy-chained interrupt sources.

Declaration

```
PUBLIC apError apVIC_PriorityDaisyGet( apOS_VIC_oId oId,
                                     UBYTE8      *pPriority )
```

Parameters

<i>old</i>	The interrupt controller to read priority for daisy-chained sources
<i>pPriority</i>	Pointer to the variable the interrupt priority to be read in

Return Value

<i>apERR_NONE</i>	Interrupt priority read, no error
<i>apERR_BAD_PARAMETER</i>	The specified controller is out of the current implementation's range

7.2.11 apVIC_IRQPreDispatchSet

Registers a function to be called when an IRQ interrupt exception occurs, but before the interrupt dispatcher (and the individual interrupt handlers) is called.

Declaration

```
PUBLIC void apVIC_IRQPreDispatchSet( apVIC_rDispatchHandler rPreDispatchHandler )
```

Parameters

rPreDispatchHandler Pointer to the function to be registered

Remarks

Registering a function supercedes any previous registered functions. If a NULL pointer is passed, the pre-dispatcher will not be called.

Note: *For this to have effect, the pre-processor constant `apOS_CONFIG_VIC_USE_PREDISPATCH_CODE` in **apintcfg.h** must have been # defined to 1.*

7.2.12 apVIC_IRQPostDispatchSet

Registers a function to be called when an IRQ interrupt exception occurs, but after the interrupt dispatcher (and the individual interrupt handlers) is called.

Declaration

```
PUBLIC void apVIC_IRQPostDispatchSet( apVIC_rDispatchHandler rPostDispatchHandler )
```

Parameters

rPostDispatchHandler Pointer to the function to be registered

Remarks

Registering a function supercedes any previous registered functions. If a NULL pointer is passed, the post-dispatcher will not be called.

Note: *For this to have effect, the pre-processor constant `apOS_CONFIG_VIC_USE_POSTDISPATCH_CODE` in **apintcfg.h** must have been # defined to 1.*

7.2.13 apVIC_ControllerDisable

Temporarily disables all interrupts for a particular interrupt controller.

Declaration

```
PUBLIC void apVIC_ControllerDisable( apOS_VIC_oId oController )
```

Parameters

oController The interrupt controller to disable

Remarks

Disables the interrupt sources for this controller in hardware (keeping a software copy for re-enabling them using `apVIC_ControllerRestore()`). All subsequent accesses to the VIC module act only on this software copy, until such time as the function `apVIC_ControllerRestore()` is called to countermand this function.

Note: *Do not nest apVIC_ControllerDisable()/apVIC_ControllerRestore() pairs. If you were to for example Disable, Disable, Restore, Restore, then the first call to apVIC_ControllerRestore() will restore the interrupts (not the second as might be expected).*

7.2.14 apVIC_ControllerRestore

Used to return to normal operation after a call to apVIC_ControllerDisable().

Declaration

```
PUBLIC void apVIC_ControllerRestore( apOS_VIC_oId oController )
```

Parameters

oController The interrupt controller to restore

Remarks

Re-enables all the interrupts which were enabled before a call to apVIC_ControllerDisable(). Copies the software version of the interrupt sources, which are enabled, to the hardware and allows the other interrupt functions to access the hardware directly again.

7.2.15 apVIC_InterruptClear

Clears VIC hardware interrupt priority logic for specified interrupt controller.

Declaration

```
PUBLIC void apVIC_InterruptClear( apOS_VIC_oId oController )
```

Parameters

oController The interrupt controller to clear priority logic

Remarks

This function should only be called at the end of Raw IRQ interrupt handlers. For other interrupt types the VIC driver clears the source.

Note: *In Daisy-chained VIC blocking mode (apOS_CONFIG_VIC_0_BLOCKING_MODE in **apintcfg.h** is # defined to 0) daisy-chained Raw IRQ interrupt handler must also call this function for all interrupt controllers that are located before this controller in the chain. In VIC 0 blocking mode (apOS_CONFIG_VIC_0_BLOCKING_MODE in **apintcfg.h** is # defined to 1) only the function for the master VIC 0 must be called.*

7.2.16 apVIC_InterruptDisable

Disables the specified interrupt source from producing an interrupt.

Declaration

```
PUBLIC void apVIC_InterruptDisable( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource The interrupt source to disable

Remarks

Writes to the interrupt controller mask to disable the specified interrupt. All other interrupt sources remain unaffected. This function does not return any errors.

7.2.17 apVIC_InterruptEnable

Enables the specified interrupt source.

Declaration

```
PUBLIC apError apVIC_InterruptEnable( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

<i>oSource</i>	The interrupt source to enable
----------------	--------------------------------

Return Value

<i>apERR_NONE</i>	The interrupt source enabled, no errors
<i>apERR_BAD_PARAMETER</i>	The interrupt source was not found or interrupt handler was not previously set (provided that compile switch apDEBUG was defined)

Remarks

Writes to the interrupt controller mask to enable the interrupt. Whether the source generates either an IRQ or FIQ should have been pre-chosen by the user by calling apVIC_PrioritySet() or apVIC_HandlerSet()/apVIC_RawHandlerSet() (otherwise the default action is to assign it to IRQ).

Always ensure that you have set an interrupt handler for a particular source before enabling its interrupt. Otherwise the system will try to detect that this situation has occurred and ignore the request to enable the interrupt.

7.2.18 apVIC_ProgrammedInterruptSet

Generates a user specified programmable interrupt. This will interrupt the processor if the interrupt selected has been enabled.

Declaration

```
PUBLIC void apVIC_ProgrammedInterruptSet( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

<i>oSource</i>	The interrupt source to set the programmed interrupt
----------------	--

7.2.19 apVIC_ProgrammedInterruptClear

Clears a previously set user specified programmable interrupt.

Declaration

```
PUBLIC void apVIC_ProgrammedInterruptClear( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource The interrupt source to clear

7.2.20 apVIC_GetInterruptStatus

Returns status of the interrupt sources for specified controller.

Declaration

```
PUBLIC UWORD32 apVIC_GetInterruptStatus( apOS_VIC_oId oId )
```

Parameters

old The interrupt controller to define interrupt status

Return Value

Interrupt status before masking

Remarks

This function defines the status of the interrupt sources for the specified vectored interrupt controller by reading the value of the raw interrupt status register. A HIGH bit indicates that the appropriate interrupt request is active before masking.

7.2.21 apVIC_SingleFIQSourceSet

Registers a source that will be marked as the only active registered interrupt source for FIQ.

Declaration

```
PUBLIC apError apVIC_SingleFIQSourceSet( CONST apOS_INT_oInterruptSource oSource )
```

Parameters

oSource The FIQ interrupt source to be registered

Return Value

<i>apERR_NONE</i>	FIQ single source registered, no errors
<i>apERR_VIC_BAD_CALL</i>	The function was called whilst interrupt is being serviced
<i>apERR_VIC_SOURCE_NOT_FOUND</i>	Interrupt source was not found
<i>apERR_VIC_BAD_INT_TYPE</i>	Interrupt type of the source is not apVIC_FIQ

Remarks

If this pointer to the source is non-zero, then this entry will always be loaded and its ISR called whenever a FIQ exception occurs. If the table lookup is required (if there is more than one FIQ handler, or if interrupt source checking is required) then set the pointer to NULL by calling apVIC_SingleFIQSourceClear() function.

Note: For this to have effect, the pre-processor constant *apOS_CONFIG_VIC_SINGLE_FIQ* in **apintcfg.h** must have been # defined to 1.

7.2.22 apVIC_SingleFIQSourceClear

Clears the source that was marked as the only active registered interrupt source for FIQ.

Declaration

```
PUBLIC void apVIC_SingleFIQSourceClear( void )
```

Parameters

None

Return Value

None

Remarks

This function sets the pointer to the source which was marked as the only active registered interrupt source for FIQ to NULL, so this entry will not be loaded and its ISR will not be called by FIQ exception.

Note: *For this to have effect, the pre-processor constant `apOS_CONFIG_VIC_SINGLE_FIQ` in ***apintcfg.h*** must have been # defined to 1.*

